

Day 01: Technology Stack Overview & Project Architecture

This document provides a comprehensive foundation for building the **CineVerse Project**, an industry-inspired full-stack application. The focus of Day 01 is to establish a **clear architectural vision**, understand the **technology stack**, and define the **system design at a conceptual level** before implementation begins.

By the end of this session, learners will not just know tools, but understand

- why those tools are used,
- how they interact, and
- what trade-offs exist in real-world systems
- Understand modern software architecture paradigms
- Analyze technology selection based on requirements
- Design a high-level scalable system architecture
- Define API contracts and system boundaries

Introduction to CineVerse Project

CineVerse is envisioned as a **movie discovery and management platform**, similar in concept to applications like Netflix or IMDb (simplified for learning).

Core Features:

- Content discovery (movies/shows)
- User authentication & personalization
- Data interaction (reviews, ratings)
- Data retrieval with performance optimization
- Scalability (horizontal expansion)
- Low latency response
- High availability
- Secure communication

Why this project?

- Covers full-stack development lifecycle
- Demonstrates real-world scalability challenges
- Integrates multiple modern technologies

Academic Insight

System design must begin with **requirement classification (functional vs non-functional)**, as architecture decisions directly depend on them.

Industry-Level Project Architecture

Modern applications are not built as a single block anymore—they are **distributed systems**.

High-Level Architecture

- Frontend → Client Interface
- API Gateway → Entry point
- Backend Services → Business logic
- Database → Persistent storage
- Cache → Performance optimization
- Message Broker → Async communication

Example Flow:

User opens app → Frontend sends request → API Gateway routes → Backend processes → Database queried → Response returned

Monolith vs Microservices

	Monolith	Microservices
--	----------	---------------

Definition	A single unified application where all functionalities are tightly coupled and run as one unit.	It divides the application into small, independent services , each responsible for a specific business function.
Characteristics	Single codebase	Independent services
	Single deployment unit	Each service has its own database
	Shared database	Communication via APIs
	Tightly coupled components	Independently deployable
Example	A single application that handles login, movie listing, booking, payments, etc.	Different applications for Authentication Service, Movie Service, Booking Service, and Payment Service
Advantages	Simple to develop initially	Independent scaling
	Easy to deploy	Faster development (parallel teams)
	Easier debugging (single codebase)	Fault isolation
		Technology flexibility
Disadvantages	Difficult to scale	Complex to manage
	Hard to maintain as it grows	Requires DevOps and containerization
	A failure in one part affects the entire system	Debugging is harder
	Slower development for large teams	Network latency issues
Development Speed	Fast Initially	Moderate
Scalability	Limited	High
Deployment	Single Unit	Independent
Fault Isolation	Low	High
Complexity	Low	High

Why Microservices for CineVerse?

The system involves **heterogeneous workloads**:

- High read operations (movie browsing)
- Moderate write operations (reviews)
- Authentication-heavy flows

Using microservices ensures:

- Independent scaling of **read-heavy services**
- Isolation of **authentication mechanisms**

- Flexibility in **technology adaptation per service**

Real-world Insight

If millions of users are watching movies:

- Only the streaming service needs scaling
- Not the entire application

Key Benefits in this Project

- Modular development
- Parallel team work
- Real-world exposure

Key Insight:

Architecture is not chosen for complexity, but for **scalability alignment with workload characteristics**.

Complete Tech Stack Overview

Layer	Technology	Purpose
Frontend	React.js	UI Development
Backend	Spring Boot	API & Business Logic
Database	PostgreSQL & MongoDB	Data Storage
Cache	Redis	Fast Access
Messaging	RabbitMQ	Async Communication
Auth	JWT	Secure Authentication
DevOps	Docker, CI/CD	Deployment & Automation

React.js Overview

React.js is a **component-based javascript frontend library**.

Key Concepts

- Components (Reusable UI blocks)
- State & Props
- Virtual DOM (efficient updates)

Example

```
function MovieCard({ title }) {  
  return <h1>{title}</h1>;  
}
```

Why React?

- Fast rendering
- Large ecosystem
- Industry standard

Spring Boot Overview

Spring Boot simplifies backend development.

Key Features

- REST API Creation
- Dependency Injection
- Embedded Server

Example

```
@GetMapping("/movies")
public List<Movie> getMovies() {
    return movieService.getAllMovies();
}
```

PostgreSQL vs MongoDB

PostgreSQL (Relational)

- Structured Data
- Strong Consistency

Example: Tables – Users, Movies, Reviews

MongoDB (NoSQL)

- Flexible schema
- JSON-like documents

Example:

```
{
  "user": "Aman",
  "watchlist": ["Movie1", "Movie2"]
}
```

When to use what?

- PostgreSQL → Transactions, relationships □ ensures ACID compliance
- MongoDB → Flexible, scalable data □ enables schema flexibility

Redis Overview

Redis is used for caching.

Why caching?

Instead of querying DB repeatedly:

- Store frequently accessed data in memory
- Reduce database load
- Improve response time

Concept

Caching follows temporal locality principle:

- Frequently accessed data should be stored closer to computation

Example

- Popular movies list cached
- Faster response time

RabbitMQ Overview

RabbitMQ enables asynchronous communication and decouples services.

Concept

Producer → Queue → Consumer

Example

- User posts review → message sent → notification service processes it

JWT Authentication Flow

JWT (JSON Web Token) ensures secure stateless authentication.

Flow

- User logs in
- Server generates token
- Token sent with every request
- Server validates token

Insight

- Stateless authentication (eliminates need for server-side session storage)
- Scalable for distributed systems

API Gateway Basics

API Gateway acts as single entry point (centralized request handler).

Responsibilities

- Routing requests
- Authentication check
- Rate limiting

Architectural Benefit

- Reduces direct exposure of services
- Simplifies client interaction

Example

Frontend → /api/movies → Gateway → Movie Service

Docker & CI/CD Introduction

Docker

Containerizes applications with dependencies.

Concept: Build once, run anywhere

Benefit: Runs same everywhere

CI/CD

Continuous Integration & Deployment

Theoretical Benefit

- Reduces integration risk
- Ensures continuous delivery

Pipeline Example: Code → Build → Test → Deploy

Project Structure (Conceptual)

```
cineverse/  
├── frontend/ (React)  
├── backend/  
│   ├── auth-service/  
│   ├── movie-service/  
│   └── review-service/  
├── gateway/  
├── docker/  
└── docs/
```

API Design (High-Level)

APIs are designed using REST principles.

Key Characteristics

- Resource-based endpoints
- Stateless communication
- Standard HTTP methods

Example Endpoints

- GET /movies
- POST /auth/login
- POST /reviews

Learning Outcomes

By the end of Day 01, learners will be able to:

- Explain differences between architectural paradigms
- Justify technology selection based on system requirements
- Illustrate high-level system architecture
- Define API endpoints at design level

Deliverables of Day 01

- Clear understanding of system architecture
- Tech stack finalized with justification
- Repository initialized
- Base folder structure created
- API endpoints designed (conceptually)

Viva Questions

1. What is the difference between monolithic and microservices architecture?
2. Why are microservices preferred in large-scale applications?
3. Explain the concept of loose coupling and high cohesion.
4. Explain the role of an API Gateway.
5. Why is React.js suitable for modern frontend development?
6. What is the Virtual DOM in React?
7. How does Spring Boot simplify backend development?
8. Compare PostgreSQL and MongoDB with use cases.
9. What is caching and why is Redis used?
10. Explain asynchronous communication using RabbitMQ.
11. What is JWT and how does authentication flow work?

12. Why is JWT considered stateless?
13. What are the responsibilities of an API Gateway?
14. What are the advantages of Docker?
15. What is CI/CD and why is it important?
16. Why is requirement analysis important before architecture design?
17. How do frontend and backend communicate in a full-stack application?
18. What are REST APIs?
19. Why is project architecture important before development?
20. Explain horizontal vs vertical scaling.
21. What are the challenges of microservices architecture?
22. How does system design impact performance and scalability?